EXERCISES LAB4.

1. Write PL/SQL code to create a function to check if a number is prime.

```
CREATE OR REPLACE FUNCTION so_nguyen_to (p_n IN NUMBER)
RETURN NUMBER
IS
   -- Gan variable i bang 2
   v_i number :=2;
BEGIN
   -- Truong hop khong phai so nguyen to
   IF p_n IS NULL OR p_n < 2 OR p_n <> TRUNC(p_n) THEN
      RETURN 0;
   END IF;

   -- Truong hop la so nguyen to
   WHILE v_i * v_i <= p_n LOOP
      IF MOD (p_n, v_i) = 0 THEN
         RETURN 0;
      END IF;
      v_i := v_i + 1;
   END LOOP;

   RETURN 1;
END so_nguyen_to;
/
```

```
CREATE OR REPLACE FUNCTION so_nguyen_to (p_n IN NUMBER)
RETURN NUMBER
IS
    -- Gan variable i bang 2
    v_i number :=2;
BEGIN
    -- Truong hop khong phai so nguyen to
    IF p_n IS NULL OR p_n < 2 OR p_n <> TRUNC(p_n) THEN
        RETURN 0;
    END IF;

    -- Truong hop la so nguyen to
    WHILE v_i * v_i <= p_n LOOP
        IF MOD (p_n, v_i) = 0 THEN
            RETURN 0;
        END IF;
        v_i := v_i + 1;
    END LOOP;

    RETURN 1;
END so_nguyen_to;
/
```

```
-- kiem tra
SELECT so_nguyen_to(7) AS p7, so_nguyen_to(9) AS p9, so_nguyen_to(1) AS p1 FROM dual;
```

Script Output | Query Result

SQL | All Rows Fetched: 1 in 0.024 seconds

	P7	P9	P1
1	1	0	0

2. Write PL/SQL code to create a procedure to calculate the factorial of a number.

```
CREATE OR REPLACE PROCEDURE factorial (
   n IN NUMBER,
   n_result OUT NUMBER
) IS
BEGIN
   -- kiem tra dau vao
   IF n IS NULL OR n < 0 OR n <> TRUNC(n) THEN
   -- dừng chương trình và ném ra một thông báo lỗi
   -- nếu gặp n không thỏa đầu vào
   RAISE_APPLICATION_ERROR(-20001,
    'Input phai la so khong am!');
   END IF;
   -- tinh giai thua
   n_result := 1;
   FOR i IN 2 .. n LOOP
      n_result := n_result * i;
   END LOOP;
END factorial;
/
```

```
CREATE OR REPLACE PROCEDURE factorial (
    n IN NUMBER,
    n_result OUT NUMBER
) IS
BEGIN
    -- kiem tra dau vao
    IF n IS NULL OR n < 0 OR n <> TRUNC(n) THEN
    -- dừng chương trình và ném ra một thông báo lỗi
    -- nếu gặp n không thỏa đầu vào
    RAISE_APPLICATION_ERROR(-20001,
      'Input phai la so khong am!');
    END IF;
    -- tinh giai thua
    n_result := 1;
    FOR i IN 2 .. n LOOP
        n_result := n_result * i;
    END LOOP;
END factorial;
/
```

```
-- kiem tra
SET SERVEROUTPUT ON;
DECLARE
    ket_qua NUMBER;
BEGIN
    -- 1. Thử nghiệm với số hợp lệ
    factorial(5, ket_qua);
    DBMS_OUTPUT.PUT_LINE('Giai thua cua 5 là: ' || ket_qua);

    -- 2. Thử nghiệm với số 0 (0! = 1)
    factorial(0, ket_qua);
    DBMS_OUTPUT.PUT_LINE('Giai thua cua 0 là: ' || ket_qua);
END;
/
```

Script Output | Query Result | Task completed in 0.023 seconds

```
Giai thua cua 5 là: 120
Giai thua cua 0 là: 1
```

3. **Write a PL/SQL package that contains a function to calculate the bonus amount for employees based on their salary and years of service.**

```
CREATE OR REPLACE PACKAGE employee_bonus AS
   FUNCTION cal_bonus(
      p_employee_id IN NUMBER
   ) RETURN NUMBER;
END employee_bonus;
/

CREATE OR REPLACE PACKAGE BODY employee_bonus AS

   FUNCTION cal_bonus(
      p_employee_id IN NUMBER
   ) RETURN NUMBER
   IS
      v_salary    EMPLOYEES.salary%TYPE;
      v_hire_date EMPLOYEES.hire_date%TYPE;
      v_years     NUMBER;
      v_bonus     NUMBER;
   BEGIN
      -- Get salary and hire date
      SELECT salary, hire_date
      INTO v_salary, v_hire_date
      FROM EMPLOYEES
      WHERE employee_id = p_employee_id;

      -- Calculate years of service
      v_years := TRUNC(
         MONTHS_BETWEEN(SYSDATE, v_hire_date) / 12
      );

      -- Calculate bonus
```

```
        IF v_years < 5 THEN
            v_bonus := v_salary * 0.05;
        ELSIF v_years < 10 THEN
            v_bonus := v_salary * 0.10;
        ELSE
            v_bonus := v_salary * 0.15;
        END IF;

        RETURN v_bonus;
    END cal_bonus;

END employee_bonus;
/
```

```
SELECT employee_bonus.cal_bonus(100)
FROM dual;
```

Script Output | Query Result 2

SQL | All Rows Fetched: 1 in 0.001 seconds

	EMPLOYEE_BONUS.CAL_BONUS(100)
1	3600

4. **Write a PL/SQL that creates a procedure to calculate the average salary of employees in a specific department.**

```
CREATE OR REPLACE PROCEDURE tinh_luong_trunh_binh (
    p_department_id IN EMPLOYEES.DEPARTMENT_ID%TYPE,
    p_trung_binh_luong OUT NUMBER
)
IS
BEGIN
    SELECT AVG(salary)
    INTO p_trung_binh_luong
    FROM employees
    WHERE department_id = p_department_id;
END tinh_luong_trunh_binh ;
/
```

```
CREATE OR REPLACE PROCEDURE tinh_luong_trunh_binh (
    p_department_id IN EMPLOYEES.DEPARTMENT_ID%TYPE,
    p_trung_binh_luong OUT NUMBER
)
IS
BEGIN
    SELECT AVG(salary)
    INTO p_trung_binh_luong
    FROM employees
    WHERE department_id = p_department_id;
END tinh_luong_trunh_binh ;
/
```

```
--kiem tra
SET SERVEROUTPUT ON;
DECLARE
    v_luong_trung_binh NUMBER;
BEGIN
    tinh_luong_trunh_binh(10, v_luong_trung_binh);

    DBMS_OUTPUT.PUT_LINE(
        'Luong trung binh: ' || v_luong_trung_binh
    );
END;
/
```

Script Output x

Task completed in 0.016 seconds

```
Luong trung binh: 4400
```

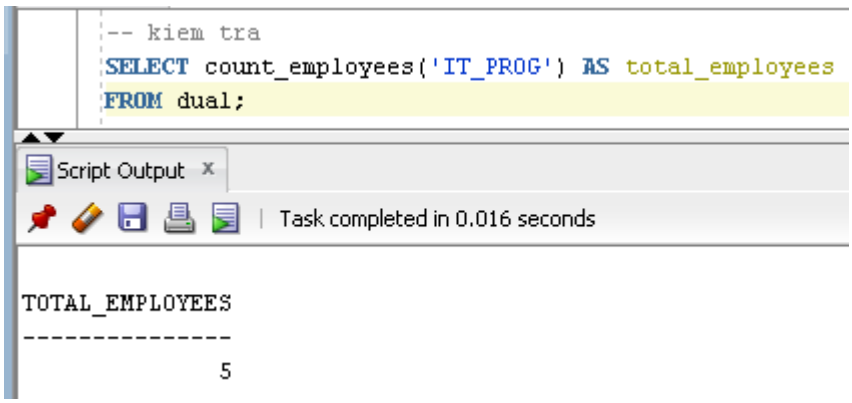
5. **Write a PL/SQL that creates a function to retrieve the total number of employees in a specific job title**

```
CREATE OR  REPLACE FUNCTION count_employees (
   p_job_id IN EMPLOYEES.JOB_ID%TYPE
)
RETURN NUMBER
IS
   v_total NUMBER;
BEGIN
   SELECT COUNT(*)
   INTO v_total
   FROM EMPLOYEES
   WHERE JOB_ID = p_job_id;

   RETURN v_total;
END count_employees;
/
```

```
CREATE OR  REPLACE FUNCTION count_employees (
    p_job_id IN EMPLOYEES.JOB_ID%TYPE
)
RETURN NUMBER
IS
    v_total NUMBER;
BEGIN
    SELECT COUNT(*)
    INTO v_total
    FROM EMPLOYEES
    WHERE JOB_ID = p_job_id;

    RETURN v_total;
END count_employees;
/
```



6. **Write a PL/SQL that creates a procedure to update an employee's salary based on their performance rating.**

```sql
CREATE OR REPLACE PROCEDURE update_employee_salary (
   p_employee_id IN EMPLOYEES.EMPLOYEE_ID%TYPE,
   p_rating      IN NUMBER,
   p_new_salary  OUT NUMBER
)
IS
   v_increase_rate NUMBER;
BEGIN
   IF p_rating < 1 OR p_rating > 5
     OR p_rating IS NULL THEN
      RAISE_APPLICATION_ERROR(
         -20001,
         'Rating must be between 1 and 5'
      );
   END IF;

   IF p_rating = 5 THEN
      v_increase_rate := 0.15;
   ELSIF p_rating = 4 THEN
      v_increase_rate := 0.10;
   ELSIF p_rating = 3 THEN
      v_increase_rate := 0.05;
   ELSE
      v_increase_rate := 0;
   END IF;

   UPDATE EMPLOYEES
   SET salary = salary * (1 + v_increase_rate)
   WHERE employee_id = p_employee_id;

   IF SQL%ROWCOUNT = 0 THEN
      RAISE_APPLICATION_ERROR(
         -20002,
         'Employee not found'
      );
   END IF;

   SELECT salary
   INTO p_new_salary
   FROM EMPLOYEES
   WHERE employee_id = p_employee_id;
```

```
END update_employee_salary;
/
```